



IRFU/ Titan-22-188

Charte Graphique pour les vues de supervision TITAN

20220530_CEA-Irfu_CharteGraphique.docx

	Rédacteur	Vérificateur	Approbateur
Nom	K.Saintin	Y. Mariette E. Bougamont D. Chirpaz O. Tuske T. Hamelin	R. Ferdinand E. Petit
Fonction	Ingénieur logiciel	Resp. WP8 Resp. Qualité Experts métier	Chef de projet TITAN Chef de projet adjoint
Date	30/05/2022	24/10/2022	02/12/2022
Visa		 E. Bougamont  	 

HISTORIQUE DES MODIFICATIONS

Version	Date	Pages modifiées	Motifs
0.1	30/05/2022	Toutes	Création du document
0.2	27/07/2022		Relecture Yannick Mariette
0.3	14/10/2022	Code couleur	Mise à jour après validation des experts
1.0	21/10/2022	Numéro Chrono	Première version signée avec numéro chrono

DIFFUSION

CEA	Externe
· E. PETIT	·
· R. FERDINAND	·
· B. POTTIN	
· V. HENNION	
· X. HANUS	
· A. DROUART	
· T. HAMELIN	
· N. CHAUVIN	
· D. CHIRPAZ	
· F. SENEÉ	
· B. BOLZON	
· E. GINER-DEMANGE	
· Y. GAUTHIER	
· O. TUSKE	
· A.C. CHAUX	·
· T. JOANNEM	·
· R. TOUZERY	·
· A. GAGET	·
· M. DESMONS	·
· Y. SAUCE	
· <u>Copie à :</u>	·
· C. VEYSSIERE	·
· P. LOTRUS	·
	·
	·
	·

TABLE DES MATIERES

1	Introduction	4
1.1	Objectif	4
1.2	Groupe de travail et Retour d'Expérience	4
1.3	Cadre	5
1.4	Documents de référence	5
1.5	Définitions et acronymes	5
2	Phoebus	6
2.1	Choix du frameworks EPICS	6
2.2	Choix de Phoebus	6
2.3	Gestionnaire d'alarmes	7
3	Base de travail et retour d'expérience	9
3.1	Retex IPHI	9
3.2	Retex vues ISEULT Muscade	10
3.3	Retex charte graphique de Spiral 2	11
3.4	Retex charte graphique de Ess	12
3.5	Retex charte graphique de SOLEIL	13
3.6	Tableau comparatif des codes couleurs	15
4	Charte graphique Titan	16
4.1	Qualité logicielle	16
4.1.1	Précision et Unité affichées dans les IOC EPICS	17
4.1.2	Limitation des scripts et codage dans les IOC EPICS	17
4.1.3	Utilisation des Macros	19
4.1.4	Utilisation des vues embarquées pour la modularité	20
4.1.5	Bandeau commun et librairie commune	21
4.1.6	Utilisation des scripts python en tant que fichier et limitation des scripts embarqués	21
4.2	Bonnes pratiques	22
4.2.1	Règles de nommage	22
4.2.2	Arborescence des fichiers	23
4.2.3	Navigation entre les vues	24
4.2.4	Informations développeur et application	25
4.3	Typographie, taille de police et résolution	25
4.3.1	Police par défaut	25
4.3.2	Résolution d'écran	26
4.4	Code couleurs	26
4.4.1	Couleur de fond des vues	27
4.4.2	Couleur de fond des champs de saisie des libellés et du texte	27
4.4.3	Code couleur des états	28
4.4.4	Code couleur des signaux tout ou rien	28
4.5	Librairie de composants	29
4.6	spécification d'une vue et processus de développement	30
4.6.1	Spécification d'une vue	30
4.6.2	Mise à jour, et gestion des versions	30
4.6.3	Demande de correction et d'évolution	30

1 Introduction

1.1 OBJECTIF

L'objectif du document est de définir une charte graphique permettant aux développeurs de réaliser des vues de supervision via le logiciel Phoebus. Ces vues ou Interface Homme Machine doivent être consistantes en terme d'ergonomie, de présentation applicative et d'homogénéité. Ce document propose plusieurs éléments aboutissant à cette cohérence, incluant :

- La qualité logicielle,
- Le code couleur,
- Le nommage des vues et des composants,
- La navigation entre les vues,
- La sécurisation des vues,
- Le calque commun à toutes les vues.

1.2 GROUPE DE TRAVAIL ET RETOUR D'EXPERIENCE

Afin d'établir cette charte graphique, un groupe de travail a été constitué et divers RETEX ont été restitués lors de 3 réunions de travail. Cette charte est donc issue des retours et de réflexions sur les vues existantes suivantes :

- Vues IPHI pour la source SILHI (EPICS)
- Vues du Vide pour les tests TITAN (EPICS)
- Vues et Charte Graphique de SPIRAL 2 au GANIL (EPICS)
- Vues et Charte Graphique de ESS en Suède (EPICS)
- Vues ISEULT (MUSCADE)
- Vues et Charte Graphique du Synchrotron SOLEIL (TANGO)

Ce groupe de travail initialement élargi à plusieurs acteurs du projet, afin de recueillir un panel d'utilisateurs pertinent allant de l'opérateur à l'expert d'un système, nous a permis de bénéficier d'un retour d'expérience concret afin d'établir certaines règles. Ce groupe a progressivement été réduit afin d'arbitrer et de statuer sur les bases de cette charte et de finaliser une première version de document.

Les acteurs sont :

- Arnaud ROGER qui a réalisé les vues de supervision du Vide de Titan.
- Didier CHIRPAZ responsable du vide sur Titan
- Robin FERDINAND, chef du projet Titan qui a participé à la mise en service de SPIRAL2
- Alexis GAGET qui travaille sur ESS et connaît leur guide de développement des IHM.
- Robert TOUZERY qui travaille sur ISEULT et a apporté son RETEX sur les vues MUSCADE.
- Katy SAINTIN qui a travaillé à SOLEIL et a présenté la charte graphique établie sur cette installation.
- Bruno POTTIN chef projet de projet IPHI et responsable du WP4
- Eric PETIT adjoint au chef de projet Titan
- Anne-Catherine CHAUVÉAU, en charge de l'assurance qualité produit.
- Yannick MARIETTE responsable WP8 dans lequel cette charte s'inscrit.

1.3 CADRE

Comme mentionné ci-dessus, la charte graphique s'applique à toutes les vues de supervisions expertes et opérationnelles développées avec Phoebus dans le cadre du projet Titan. Mais il pourra servir de base également pour les schémas intégrés dans les divers documents du projet Titan. Notamment pour le code couleur.

Ces indications sont à respecter dans la mesure du possible, mais certaines libertés pourront être prises en particulier pour les vues expertes. Cependant, il est important de souligner que le suivi du code couleur reste primordial et évitera des confusions possibles pour les opérateurs et les utilisateurs étant amenés à interagir avec les vues de supervision. En cas de doute contacter l'auteur du document ou le responsable projet.

Cette charte graphique ne s'applique pas à la partie expérimentale du projet, les applications et vues de cette partie seront à la charge des équipes projets responsables des dispositifs expérimentaux installés fin de faisceau.

1.4 DOCUMENTS DE REFERENCE

- [R1] Guide de développement des IHM applicatives pour ESS
- [R2] Charte graphique des applications au Synchrotron SOLEIL
- [R3] Charte graphique des vues de supervisions pour l'opération sur Spiral2
- [R3] Manuel de référence ANIBUS pour le développement des vues MUSCADE
- [R4] [Règle de nommage Titan](#) (Naming Convention)

1.5 DEFINITIONS ET ACRONYMES

CSS	CS-Studio ou Control System Studio version logicielle antérieure de Phoebus
EPICS	Experimental Physics and Industrial Control System
ESS	European Spallation Source
GUI	Graphical User Interface
IHM	Interface Homme Machine (GUI Graphical User Interface en Anglais)
IPHI	Injecteur de Protons de Haute Intensité
IOC	Input Output Controller, correspond au programme serveur EPICS
Macro	Principe de paramétrage de vues au travers de variables partagées et centralisée
LT	Ligne de Transport
PV	Process Variable , Variables EPICS
SCADA	Supervisory Control And Data Acquisition
VF	Vide Faisceau

2 Phoebus

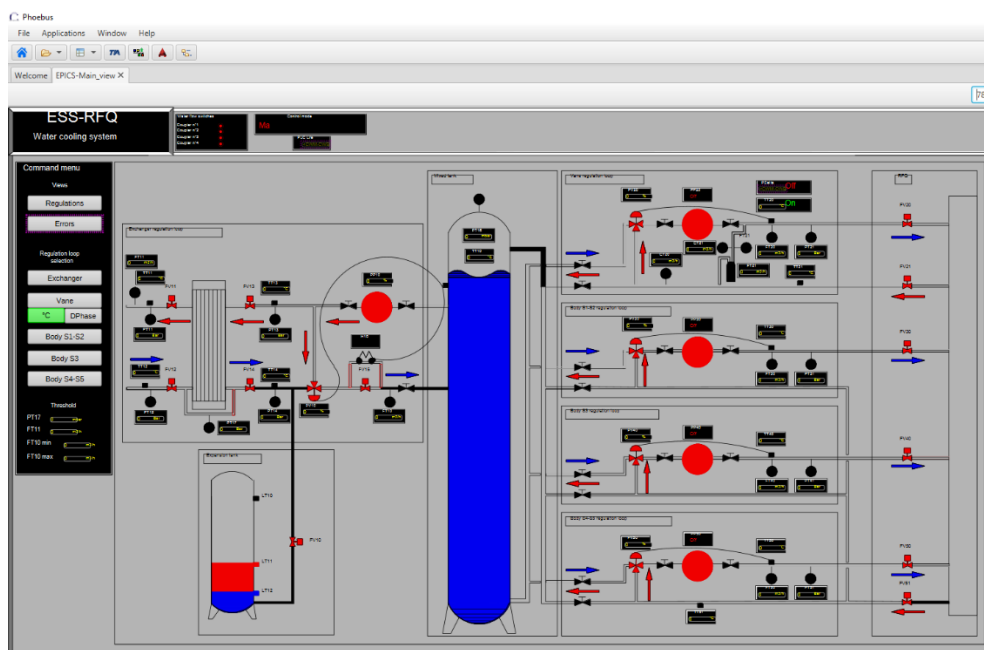
2.1 CHOIX DU FRAMEWORK EPICS

Phoebus fait partie du framework EPICS qui a été choisi pour le contrôle commande de Titan. EPICS est un ensemble d'outils logiciels, de bibliothèques et d'applications Open Source permettant de fournir une infrastructure logicielle pour la création des systèmes de contrôle d'instruments scientifiques tels que des accélérateurs de particules, des télescopes et des expériences. Initié par deux laboratoires américains (Los Alamos et Argonne) en 1986, EPICS est devenu au fil du temps une solution internationale utilisée par plus de 150 laboratoires dans le monde entier. Utiliser EPICS permet d'avoir des systèmes de contrôle distribués, interconnectés en temps réel et permet une intégration homogène des logiciels.

2.2 CHOIX DE PHOEBUS

A ce jour les écrans de supervisions connectés à EPICS à l'IRFU et les projets d'envergure tel que SARAF et ESS, sont réalisés avec CSS (Control System Studio). CSS est basé sur Eclipse RCP (Rich Client Platform), la solution est éprouvée et fiable et fortement employée sur d'autres projets EPICS. Cependant, les développeurs de CSS concentrent leur effort de développement sur un logiciel équivalent appelé Phoebus destiné à remplacer CSS, cette solution offre l'avantage de ne pas être dépendant de l'OS et se construit facilement. Le menu applicatif de Phoebus est également plus épuré que celui proposé dans CSS et rend l'IHM plus intuitive et ergonomique. Les recherches et l'analyse des solutions sous Phoebus, nous permettent de valider que les développements se feront sous Phoebus directement.

Phoebus est issue d'une collaboration entre plusieurs laboratoires et universités (SNS aux Etats-Unis à Knoxville dans le Tennessee, DESY en Allemagne à Hambourg, ESS en Suède à Lund, ITER en France à Manosque). Il est donc largement utilisé pour des accélérateurs et également bien documenté et maintenu. Le principe de Phoebus est basé sur le développement graphique par glisser/déposer. Il est ainsi accessible à des non développeurs qui souhaiteraient mettre en place une IHM rapidement. Ce dernier intègre une collection d'outils dans dont les services sont fournis par le framework EPICS.



Ecran de supervision du Skid ESS avec Phoebus

2.3 GESTIONNAIRE D'ALARMES

Phoebus intègre un gestionnaire d'alarmes imposant son propre code couleur. Côté client, les alarmes peuvent être visualisées et acquittées dans une perspective dédiée. Cette dernière affiche les alarmes sous forme d'arborescence géographique (Area) et d'une table des alarmes courantes, la sévérité d'une alarme est représentée par une couleur, une sévérité composite est calculée pour chaque zone. Un opérateur peut depuis cette interface acquitter une alarme afin qu'elle repasse en statut normal.

The screenshot displays the 'Alarm Tree' and 'Alarm Table' in the Phoebus interface. The 'Alarm Tree' on the left shows a hierarchical structure of areas, including BeamPermit, RF, Water_Pump, HEBT, SCL, RID, Ring, RTBT, Target, Instrument Hall, CER, Vacuum, CHL, Operations, HVCM, RF Transmitters, Momentum Dump, Test HVCM, In Test Facility, RF Transmitter, Once Test Stand, Klystron Gallery, BT Service Bus, and Service Bus. The 'Alarm Table' on the right shows a list of current and acknowledged alarms. The 'Current Alarms (2)' table includes columns for Description, Alarm Time, Current Severity, and Current Status. The 'Acknowledged Alarms (13)' table includes columns for Description, Alarm Time, Current Severity, and Current Status. A context menu is open over the 'Check Ring DI Water Cooling Loop' alarm, showing options like Acknowledge, Configure Item, and Disable Alarms.

Perspective Alarme dans Phoebus

Ce gestionnaire propose également des rapports statistiques sur le nombre d'occurrences d'une alarme.



Statistiques sur les alarmes

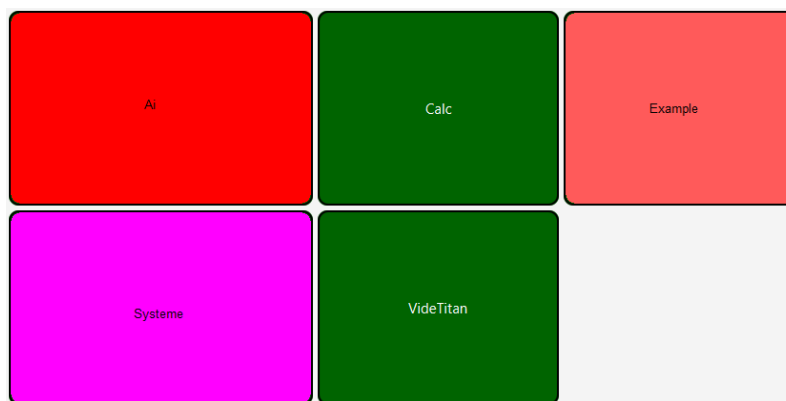
Le **code couleur et l'agencement en arborescence n'est pas configurable et est imposé par l'outil**. Nous avons donc proposé par la suite, un code couleur s'en rapprochant au maximum, tout en prenant en compte les RETEX des divers experts du groupe de travail.

Sur le forum technique, le développeur de Phoebus explique les raisons pour lesquelles ces couleurs ne sont pas personnalisables. <https://epics.anl.gov/tech-talk/2022/msg01049.php>
Il est important d'avoir un code couleur prédéfini pour les alarmes, afin que les utilisateurs sachent quelle couleur correspond à quel état d'alarme. Et voulant éviter des confusions d'un outil à l'autre.

Code couleur figé du gestionnaire d'alarme Phoebus

<https://github.com/ControlSystemStudio/phoebus/blob/master/app/alarm/ui/src/main/java/org/phoebus/applications/alarm/ui/AlarmUI.java>

Etats	Couleur du texte selon la sévérité	Couleur de fond des zones selon la sévérité
OK	0,100,0	
MINOR_ACK	120, 90, 10	180, 170, 70
MAJOR_ACK	100, 0, 0	255, 90, 90
INVALID_ACK	100, 50, 100	255, 128, 255
UNDEFINED_ACK	100, 50, 100	255, 128, 255
MINOR	207, 192, 0	207, 192, 0
MAJOR	255, 0, 0	255, 0, 0
INVALID	255, 0, 255	255, 0, 255
UNDEFINED	255, 0, 255	255, 0, 255



Selected	PV	Description	Timestamp	Value	Alarm
☒	titantest:xxxExample	xxx record	2022-07-27 18:16:16.726870100	0 Counts	INVALID/UDF_ALARM
☒	titantest:aiExample	Analog input	2022-07-27 18:14:45.022882604	9 Counts	MAJOR/HIGH_ALARM
☒	titantest:aiExample1	Analog inpu...	2022-07-27 18:24:10.023176337	4 Counts	MINOR/LOW_ALARM
☒	titantest:aiExample2	Analog inpu...	2022-07-27 18:24:10.023165992	7 Counts	MINOR/HIGH_ALARM
☒	titantest:aiExample3	Analog inpu...	2022-07-27 18:24:06.023127530	0 Counts	MAJOR/LOLO_ALARM
☒	titantest:ai1	Analog inpu...	2022-07-27 18:24:10.023176337	4 Counts	MINOR/LOW_ALARM
☒	titantest:ai3	Analog inpu...	2022-07-27 18:16:46.023075222	2 Counts	MAJOR/LOLO_ALARM
☒	titantest:ai2	Analog inpu...	2022-07-27 18:24:10.023165992	7 Counts	MINOR/HIGH_ALARM
☒	titantest:calcExample	Counter	2022-07-27 18:24:10.023168710	4 Counts	MINOR/LOW_ALARM
☒	titantest:calcExample1	Counter No. 1	2022-07-27 18:14:09.022950577	3 Counts	MINOR/LOW_ALARM
☒	titantest:calcExample2	Counter No. 2	2022-07-27 18:24:10.023159380	7 Counts	MINOR/HIGH_ALARM
☒	titantest:calcExample3	Counter No. 3	2022-07-27 18:24:06.023121084	0 Counts	MAJOR/LOLO_ALARM
☒	titantest:calc1	Counter No. 1	2022-07-27 18:24:10.023174514	4 Counts	MINOR/LOW_ALARM
☒	titantest:calc2	Counter No. 2	2022-07-27 18:24:10.023159380	7 Counts	MINOR/HIGH_ALARM
☒	titantest:calc3	Counter No. 3	2022-07-27 18:24:06.023121084	0 Counts	MAJOR/LOLO_ALARM
☒	titantest:compressExample	Circular buff...	2022-07-27 18:24:10.023234087	5.0, 6.0, ...	

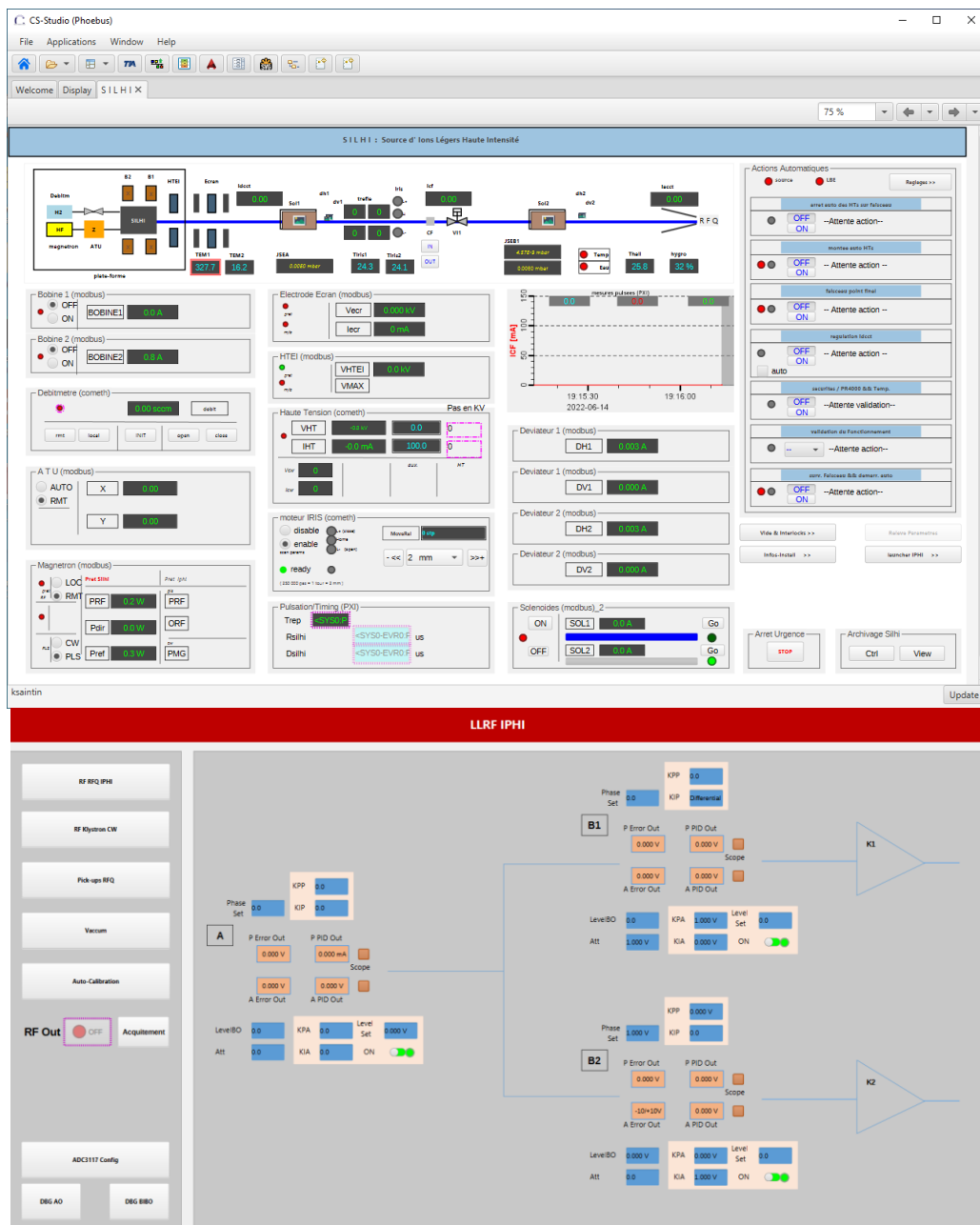
Gestionnaire d'alarmes dans Phoebus et code couleur figé

3 Base de travail et retour d'expérience

Nous avons constitué le groupe de travail afin de recueillir un maximum d'informations issues des RETEX de projets équivalents. La charte graphique décrite par la suite tire profit des retours de nos experts cités précédemment.

3.1 RETEX IPHI

La typologie et les règles de couleurs de fond des champs numériques (en lecture ou écriture) et des vues ont été inspirées par les vues IPHI existantes pour la source SILHI et le Low Level RF. L'apparition d'un bandeau visible en haut des vues a également été repris pour la Charte graphique de Titan.

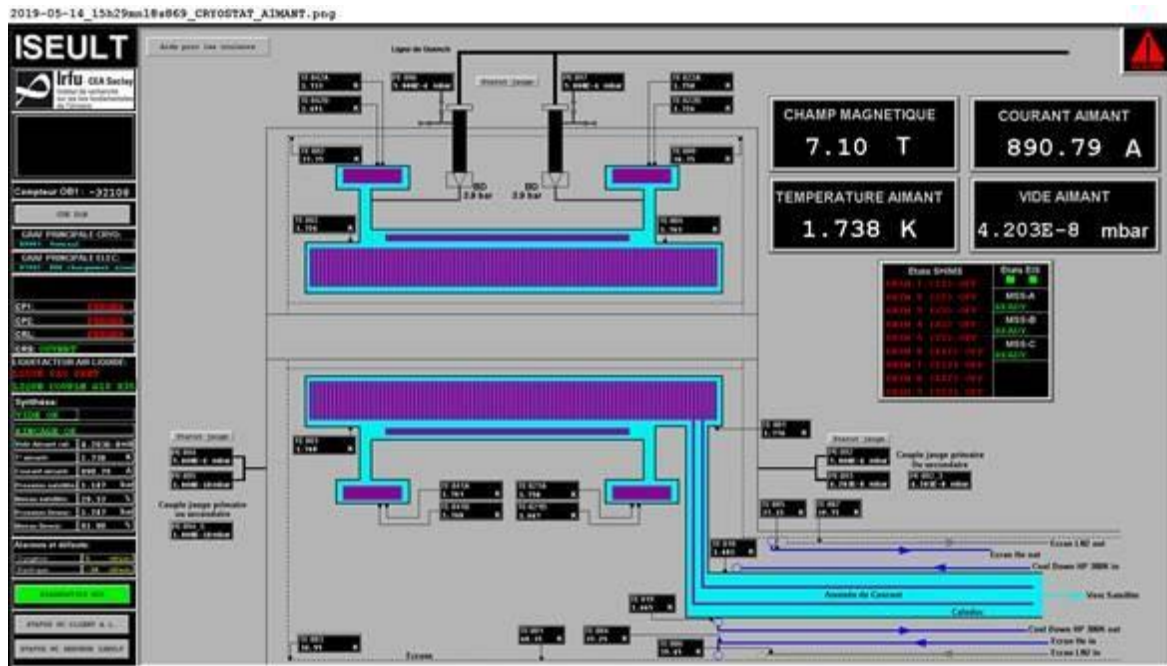


Exemple de vues pour IPHI

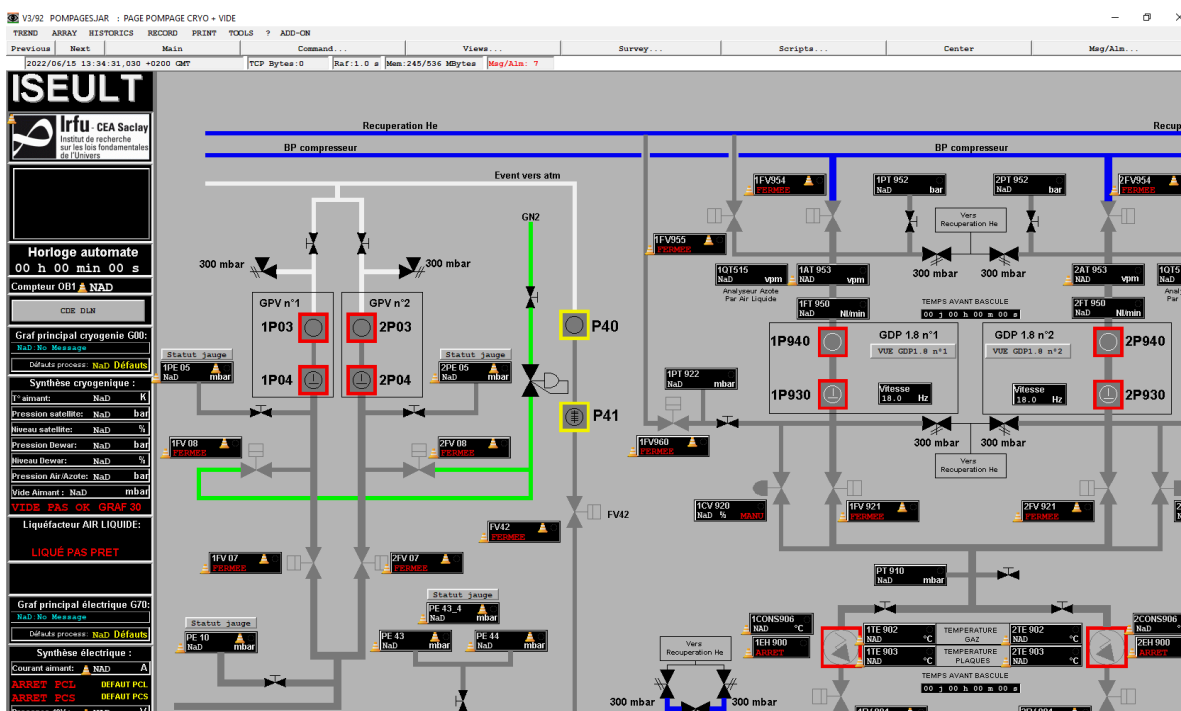
3.2 RETEX VUES ISEULT MUSCADE

Le **calque commun** incluant le **nom**, le **logo** et le **domaine d'expertise** du projet, ainsi que l'affichage des valeurs importantes dans un **bandeau récapitulatif** seront repris pour la charte graphique du projet Titan.

A cela s'ajoute les notions de **sécurité d'accès** au serveur Muscade par le biais de certificat et une authentification forte. Ainsi dans la charte graphique, ces notions seront spécifiées au travers de compte utilisateur et des droits d'accès à certaines vues ou champs.



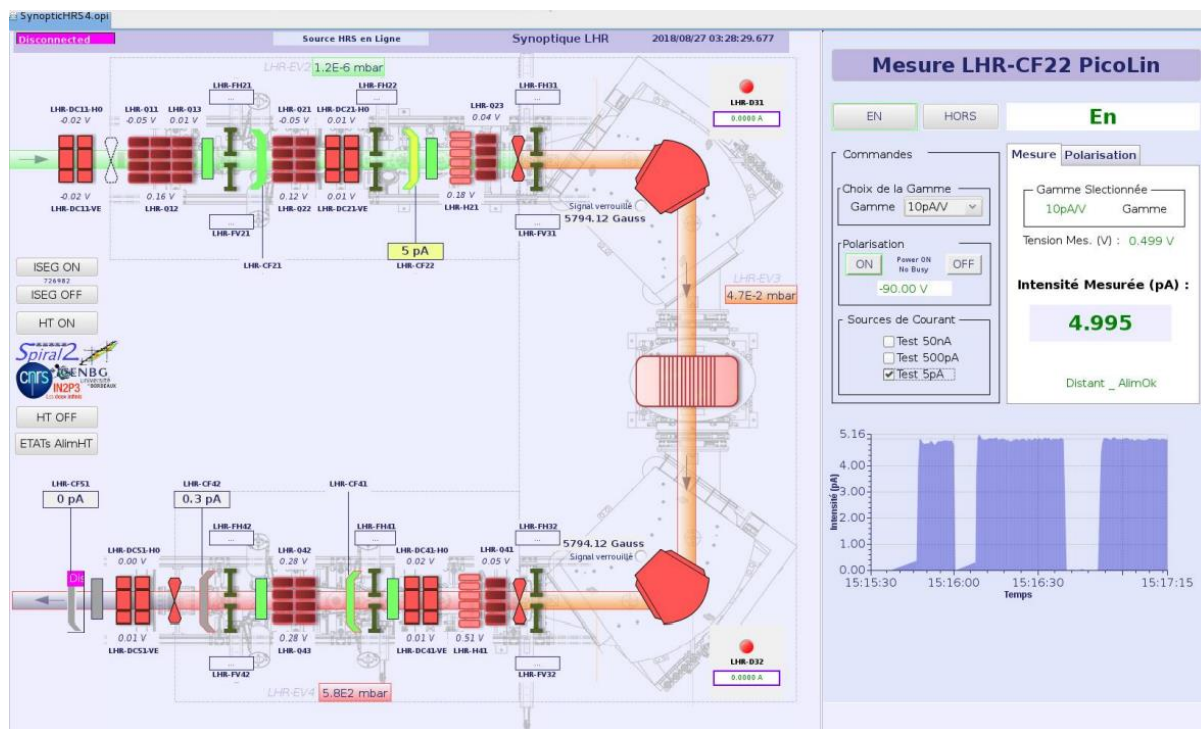
Vue Cryostat Aimant ISEULT



Vue de Pompage CRYO Vide ISEULT

3.3 RETEX CHARTE GRAPHIQUE DE SPIRAL 2

La charte graphique de SPIRAL2 décrite dans un diaporama, nous a servi également de base pour le code couleur et les bonnes pratiques. Ce dernier donne également des éléments à configurer du côté EPICS comme par exemple l'unité, la précision qui sont visibles sur les vues (cette configuration du côté du serveur EPICS répond aux critères de qualité logicielle décrite au chapitre 4.1 de ce document).



Vue Spiral 2 sous CSS

Couleurs

Couleurs primaires			Couleurs secondaires		
	Display	HEX RGB		Display	HEX RGB
Main Background		EEEEFF R=238, G=238, B=255	Header Foreground		242767 R=36, G=39, B=103
Sub Background Blue		E1E6FA R=225, G=230, B=250	Secondary 2		1E2478 R=30, G=36, B=120
Sub Background Purple		E1DEEE R=225, G=222, B=238	Secondary 3		726B8F R=114, G=107, B=143
Sub Background White		FFFFFF R=255, G=255, B=255	Secondary 4		958DB4 R=149, G=141, B=180
Primary 1		19197A R=25, G=25, B=122	Secondary 5		968FB6 R=150, G=143, B=182
Primary 2		A020F0 R=160, G=32, B=240	Secondary 6		877EAC R=135, G=126, B=172
LINK FOREGROUND		4169FF R=65, G=105, B=255	Secondary 7		A39DC3 R=163, G=157, B=195
Primary 3		2F5CB4 R=47, G=92, B=180	Secondary 8		A49BCE R=164, G=155, B=206
Primary 4		39698A R=57, G=105, B=138	Header Background		BDB5E0 R=189, G=181, B=224
Dark Grey Border		BEBEBE R=190, G=190, B=190	Secondary 11		C8C1E8 R=200, G=193, B=232
White		FFFFFF R=255, G=255, B=255	Secondary 12		BDB8E0 R=189, G=184, B=224
Alarmes			Etats Equipements		
Severity 1/ Default		FF0000 R=255, G=0, B=0	Disabled Background		E8E8E8 R=232, G=232, B=232
Severity 2/ Warning		FF8000 R=255, G=128, B=0	Disabled Foreground		8E8F91 R=142, G=143, B=145
Severity 3/ Keep in mind		FFFF00 R=255, G=255, B=0	Read Only Ttext		005000 R=0, G=80, B=0
Disconnected		FF00FF R=255, G=0, B=255	Démarré ou Hors		7AFF67 R=122, G=255, B=103
All is right		00B050 R=0, G=176, B=80	Arrêté		FF5555 R=255, G=85, B=85
Invalid		8000FF R=128, G=0, B=255	En		E9FF52 R=233, G=255, B=82
			NiNi		A3A3A3 R=163, G=163, B=163

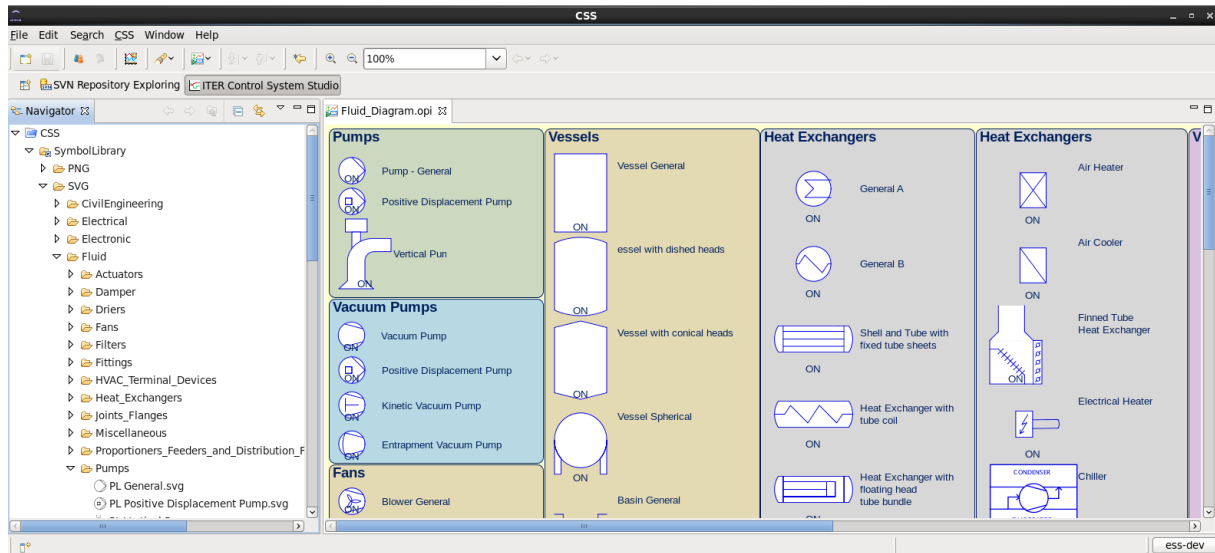
Laurent PHILIPPE – SDA/GIM – 23/08/2012

GANIL
Mesures Control Systems

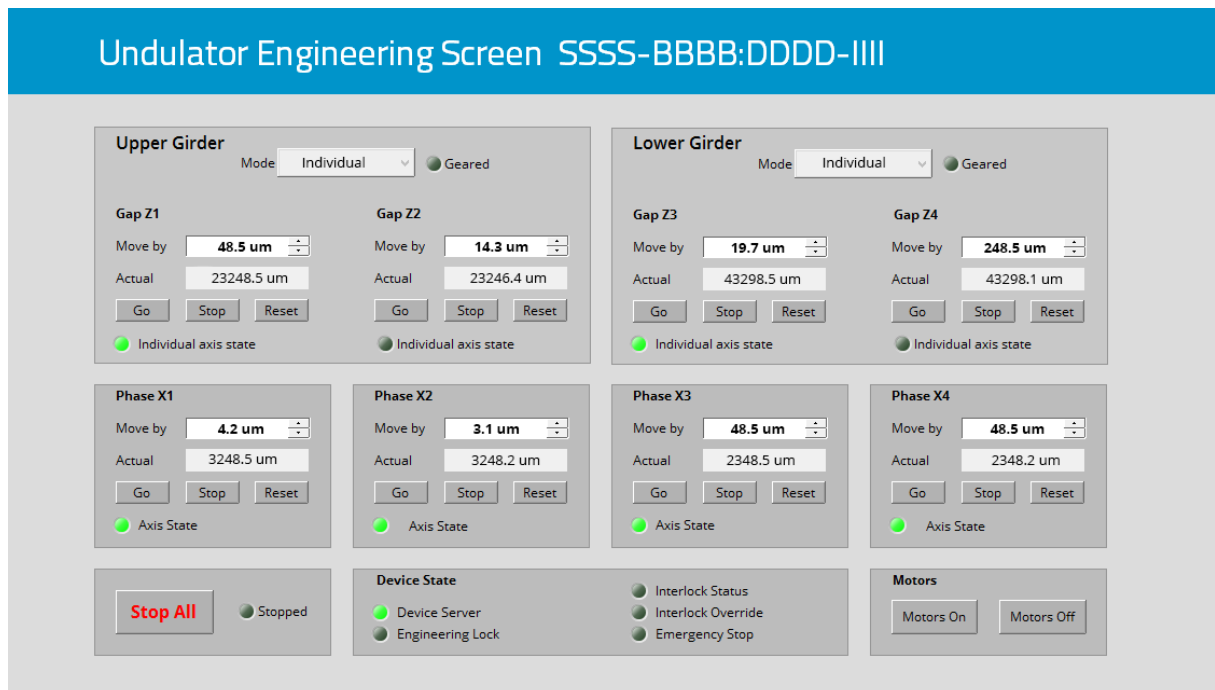
Code couleurs de la charte graphique de Spiral 2

3.4 RETEX CHARTE GRAPHIQUE DE ESS

ESS a établi un Guide de développement des Applications qui a également servi de base pour la Charte Graphique de Titan. Il propose par exemple une librairie de symboles ainsi que des recommandations pour les vues métiers.



Librairie de symboles ESS



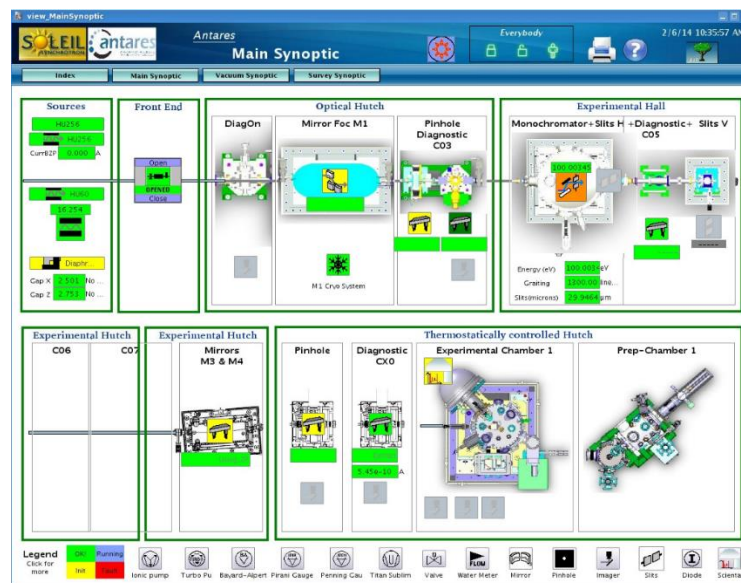
Vue métier Ondulateur ESS

3.5 RETEX CHARTE GRAPHIQUE DE SOLEIL

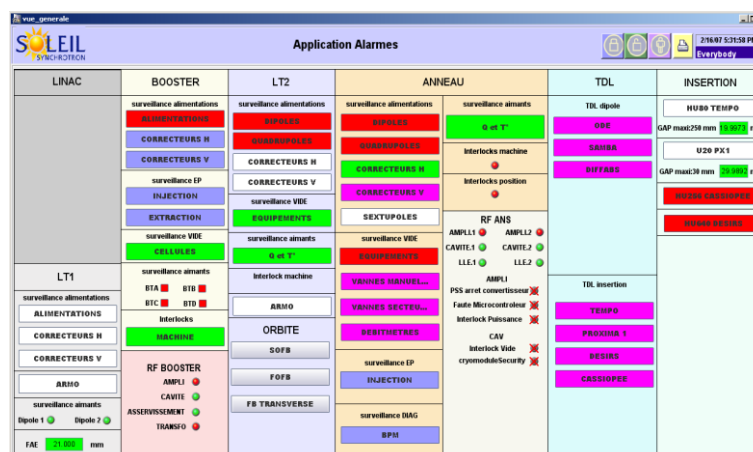
Un ensemble de documents faisant partie de la charte graphique à SOLEIL a servi de base pour le développement des vues pour la source comme pour les lignes de lumière du synchrotron. Toutes les applications sont donc homogènes sur l'ensemble du système. Le code couleur est défini dans le contrôle commande sous-jacent à savoir TANGO, et ce dernier est **fixé et interchangeable**. Ce dernier point a fortement contribué à l'homogénéité des applications de SOLEIL. Cette charte et les documents produits ont permis de réduire le coût et le temps de développement. De plus ces indices de suivi diminuaient au développement de chaque nouvelle application grâce aux RETEX réalisés régulièrement. (Méthode AGILE).

La charte graphique contient tous les éléments précédemment énoncés :

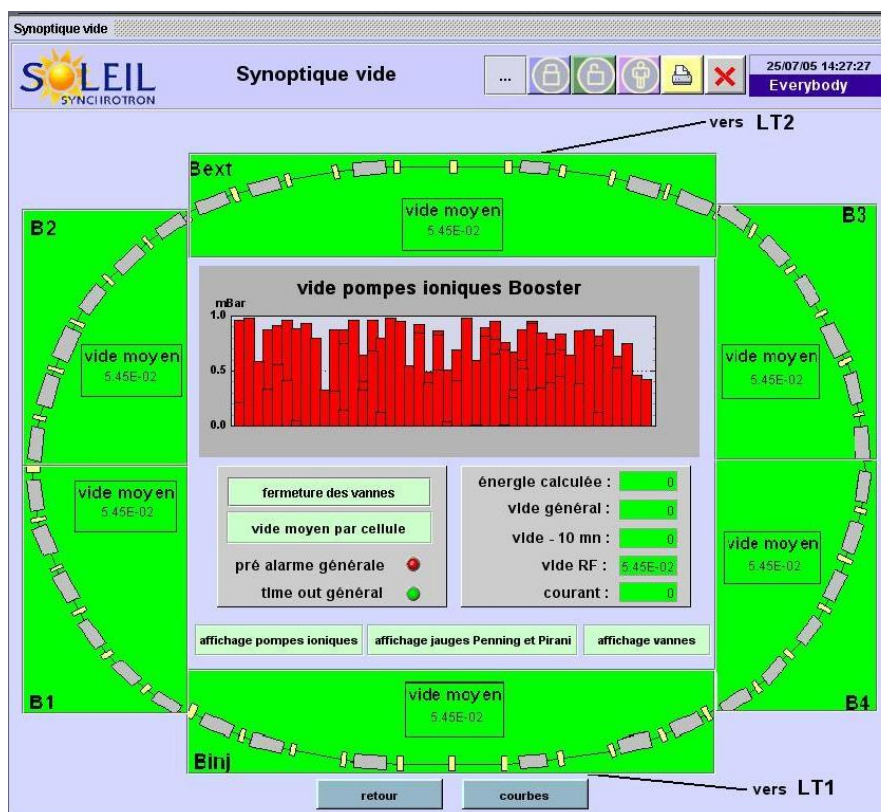
- Code couleur et typographie
- Nom de projet,
- Nomenclature des variables, des domaines et des projets
- Calque commun incluant le logo, le nom du projet
- Sécurisation des vues expertes (accès en écriture ou lecture seule)
- Librairie de symboles
- Navigation entre les vues
- Règles et recommandation de développement
- Document pour la spécification des vues avant implémentation



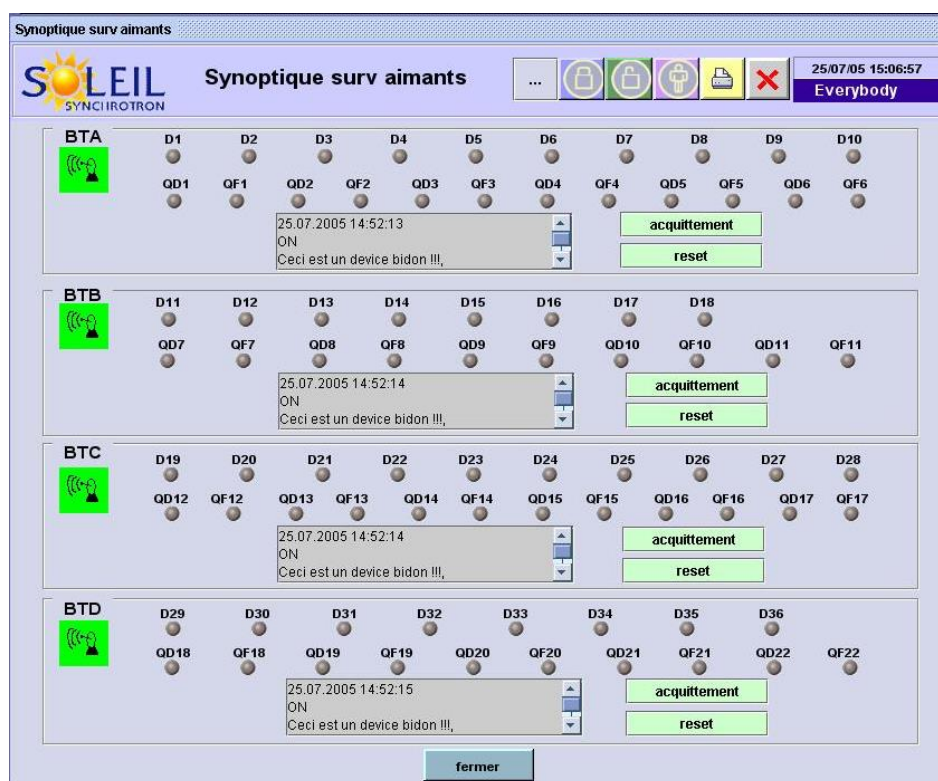
Vue principale d'une ligne de lumière réalisée par le responsable de la ligne



Vue Alarmes dans la salle de contrôle opérateur réalisée par les opérateurs



Vue de la surveillance du vide réalisé par les experts du vide.



Vue de surveillance des aimants réalisé par les experts aimants

3.6 TABLEAU COMPARATIF DES CODES COULEURS

A l'issue de tous ces retours d'expérience, il est important de statuer sur un code couleur concernant les différents états du système. Une analyse comparative a été réalisée en amont afin d'aider les experts qui exploiteront les vues majoritairement à statuer.

On différenciera le code couleur pour les états et la couleur des LED pour des signaux binaires (actif / inactif).

Etats	SOLEIL	DACM LEDA	SPIRAL 2 Vue métier	SPIRAL 2 Vue opérateur	Alarm GUI CSS-Phoebus Couleurs figées	ESS	Consensus
MARCHE	Vert	Vert	Vert	Vert	Vert	Vert	OK
ARRÊT	Blanc	Rouge	Rouge	Rouge	Vert sombre		
FERMÉ	Blanc	Rouge	Rouge	Jaune	Vert sombre		
OUVERT	Vert	Vert	Vert	Vert	Vert		OK
EN	Blanc	Rouge	Rouge	Jaune	Vert sombre		
HORS	Vert	Vert	Vert	Vert	Vert		OK
DÉFAUT	Rouge	Orange		Rouge	Rouge	Rouge	
ALARME	Orange		Vert ?	Rouge Orange Jaune	Rouge Orange Jaune	Orange	
DECONNECTÉ	Gris	Violet		Magenta	Magenta		
EN MOUVEMENT	Bleu	Bleu		Gris			
EN COURS	Bleu						
DÉSACTIVÉ	Magenta			Gris	Vert sombre		
VEILLE	Jaune						
INITIALISATION							

Comparatif sur les codes couleurs des états

Type de signal => LED	Spiral 2	ESS	IPHI	Muscade	SOLEIL	Consensus
ACTIF	Vert	Vert	Vert	Vert	Vert	OK
INACTIF	Vert sombre	Vert sombre	Vert sombre	Gris	Gris	
MARCHE	Vert	Vert	Vert	Vert	Vert	OK
ARRÊT	Vert sombre	Vert sombre	Vert sombre	Gris	Gris	
FERMÉ	Vert sombre	Vert sombre	Vert sombre	Gris	Gris	
OUVERT	Vert	Vert	Vert	Vert	Vert	OK
EN	Vert sombre	Vert sombre	Vert sombre	Gris	Gris	
HORS	Vert	Vert	Vert	Vert	Vert	OK
ALARME ou ERREUR active	Rouge	Rouge	Orange	Jaune	Rouge	
Pas d'ALARME ou d'ERREUR	Vert sombre	Vert sombre	Vert sombre	Gris	Gris	

Comparatif sur les codes couleurs des signaux binaires

4 Charte graphique Titan

La charte graphique Titan a été déclinée dans un diaporama nommée « **Charte Graphique Titan.pptx** » qui reprend tous les éléments décrits dans ce document. Ce diaporama permet également la spécification d'une vue en amont du développement sous Phoebus. Une fois une vue spécifiée grâce à ce diaporama, il pourra être remis à un développeur Phoebus qui ne nécessite pas de compétence en programmation. Cette spécification pourra également servir de base dans les échanges avec les utilisateurs finaux des applications de supervisions (opérateurs et experts).

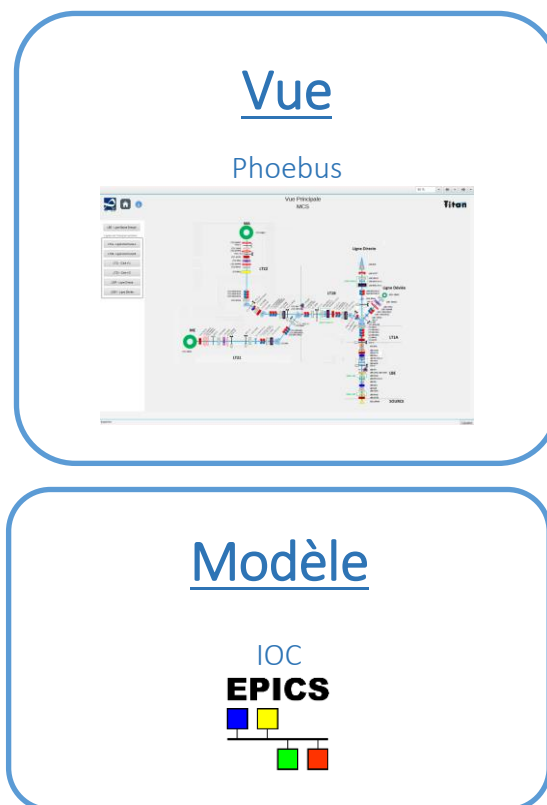
4.1 QUALITE LOGICIELLE

Afin de rendre la supervision maintenable et fiable, il est nécessaire que la complexité et l'intelligence métier soient codées au maximum dans les IOC (serveur EPICS). En qualité logicielle, cela répond à un modèle de conception très employé dans le monde du développement d'application de haut niveau. Le modèle MVC (Modèle Vue Contrôleur). Le principe de cette architecture étant de bien établir une séparation entre le code concernant le métier et le code concernant l'IHM (Interface Homme Machine).

Le **modèle** contient toute l'intelligence métier incluant les drivers et les programmes gérant les processus métiers.

La **vue** contient tout le code graphique incluant le choix du logiciel employé pour le développement des applications graphiques. Dans notre cas Phoebus.

Ce modèle de conception a fait ses preuves dans le monde du développement logiciel et permet de livrer des applications fiables et évolutives. D'une part pour la maintenance en cas d'incident ou la détection de bug, il permet aux personnes en charge du support de se concentrer directement vers les programmes métiers car l'application ne possède aucune d'intelligence métier. D'autre part en cas d'évolution du côté IHM, (changement logiciel ou autre) la partie métier pourra être conservée.



Modèle MVC en qualité logicielle appliquée au contrôle commande de Titan

4.1.1 Précision et Unité affichées dans les IOC EPICS

Pour répondre au modèle MVC, l'unité et la précision des valeurs (nombre de chiffre après la virgule) seront définies dans les IOC EPICS. Dans les fichiers de configuration EPICS, il s'agira de définir les champs EGU et PREC. Puis dans le logiciel Phoebus, il s'agira de cocher a case Unit from PV et Prec from PV. Voir l'exemple ci-dessous :

Fichier EPICS `iocTitan_vide.db` extrait du vide de Titan :

```
record(ai, "VF10:PI1:MesHVR"){  
    field(EGU, "mBar")  
    .field(PREC, "2")  
    field(DTYP, "asynInt32")  
    field(INP, "@asyn(FAST,361)INT")  
    field(SCAN, "I/O Intr")  
    field(DESC, "VF10: PI1 - Pompe Ionique - Mesure 'High")  
}
```

Définition de l'unité en mBar (millibar) pour la pression haute de la pompe ionique PI1
Définition d'une précision de 2 chiffres après la virgule concernant la précision.

Pression	13.00 mBar
----------	------------

Affichage de l'unité et de la précision sous Phoebus défini dans l'IOC

4.1.2 Limitation des scripts et codage dans les IOC EPICS

Toujours pour répondre au modèle MVC, les signaux ou valeurs calculées à partir d'autres PV seront programmés du côté IOC EPICS et publiés dans des PV EPICS. Ainsi on limitera le développement de scripts du côté client, rendant ainsi leur maintenance et leur fiabilité faciles.

Par exemple, l'utilisation des champs ONAM et ZNAM (respectivement One Name et Zero Name) pour définir un texte associé à une valeur d'une variable booléenne, plutôt que de faire un script.

Par exemple pour une Vanne extrait du fichier EPICS Vide de Titan :

```
record(bi, "VF10:GTM1-VEA:OuvR"){  
    field(DTYP, "asynUInt32Digital")  
    field(INP, "@asynMask(FAST,54,0x0200)BOOL")  
    field(ZNAM, "Vanne Fermée")  
    field(ONAM, "Vanne Ouverte")  
    field(SCAN, "I/O Intr")  
    field(DESC, "VF10: GTM1 - Vanne Entrée Azote - ouvert")  
}
```

Lorsque la valeur de la PV est à 1, la valeur textuelle correspondante sera égale à « Vanne Ouverte »
Lorsque la valeur de la PV est à 0, la valeur textuelle correspondante sera égale à « Vanne Fermée »

Vanne	Vanne fermée
	VF10:GTM1-VEA:OuvR Vanne fermée, 2022-06-29 17:35:54.015000000

Affichage des champs ZNAM sous Phoebus défini dans l'IOC

Vanne	Vanne Ouverte
	VF10:GTM1-VEA:OuvR Vanne Ouverte, 2022-06-29 17:35:54.015000000

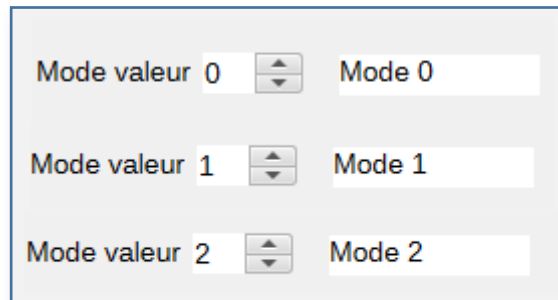
Affichage des champs ONAM sous Phoebus défini dans l'IOC

Lorsque les variables analogiques de type énumérées, on définira les champs EPICS suivants :

ZRST	Valeur textuelle lorsque la variable est égale à 0 (Zero string)
ONST	Valeur textuelle lorsque la variable est égale à 1 (One string)
TWST	Valeur textuelle lorsque la variable est égale à 2 (Two string)
THST	Valeur textuelle lorsque la variable est égale à 3 (Three string)
FRST	Valeur textuelle lorsque la variable est égale à 4 (Four string)
FVST	Valeur textuelle lorsque la variable est égale à 5 (Five string)
SXST	Valeur textuelle lorsque la variable est égale à 6 (Sixt string)
SVST	Valeur textuelle lorsque la variable est égale à 7 (Seven string)
EIST	Valeur textuelle lorsque la variable est égale à 8 (Eight string)
NIST	Valeur textuelle lorsque la variable est égale à 9 (Nine string)
TEST	Valeur textuelle lorsque la variable est égale à 10 (Ten string)
ELST	Valeur textuelle lorsque la variable est égale à 11 (Eleven string)
TVST	Valeur textuelle lorsque la variable est égale à 12 (Twelve string)
TTST	Valeur textuelle lorsque la variable est égale à 13 (Thirteen string)
FTST	Valeur textuelle lorsque la variable est égale à 14 (Fourteen string)
FFST	Valeur textuelle lorsque la variable est égale à 15 (Fifteen string)

Exemple d'une variable logique pour la définition d'un mode de fonctionnement

```
record(ao, "Mode"){
  field(ZRST, "Mode 0")
  field(ONST, "Mode 1")
  field(TWST, "Mode 2")
  field(THST, "Mode 3")
  field(FRST, "Mode 4")
}
```



Affichage des valeurs énumérées sous Phoebe définies dans l'IOC

Utilisation des variables CALC voir la documentation EPICS à ce sujet

<https://epics.anl.gov/EpicsDocumentation/AppDevManuals/RecordRef/Recordref-14.html>

Les variables de type CALC permettent de réaliser des calculs entre variables et de publier le résultat dans une variable. En définissant un maximum de ce type de variables dans l'IOC, l'IHM restera simple et limitera l'utilisation des scripts à maintenir du côté client. Calc propose diverses fonctions telles que la somme, la moyenne et l'affectation conditionnelle ...

Dans l'exemple qui va suivre (affectation conditionnelle), on publie dans une variable calcExemple la valeur de la variable ProcessVar1 + 1 si la valeur ProcessVar1 est inférieur à 9, sinon on publie 0.

```
record(calc, "calcExemple")
{
  field(CALC, "(A<B)?(A+C):D")
  field(INPA, "ProcessVar1.VAL NPP NMS")
  field(INPB, "9")
  field(INPC, "1")
  field(INPD, "0")
  field(EGU, "Counts")
}
```

Le framework offre également d'autres solutions pour implémenter des algorithmes plus complexes du côté serveur. Entre autre il y a les sous-routines qui permettent de faire appel à du code C, python ou shell.

<https://epics.anl.gov/EpicsDocumentation/AppDevManuals/RecordRef/Recordref-40.html>.

Et également la librairie SNL (State Notation Language - https://epics.anl.gov/docs/USPAS2014/2-Tuesday/SNL_2_Sequencer.pdf) qui permet de définir des séquences.

4.1.3 Utilisation des Macros

Afin de factoriser au maximum le paramétrage des vues et permettre une maintenance efficace. Il est recommandé d'utiliser des Macros. Ces Macros à définir une seule fois dans les vues, répercutent de manière centralisée leur valeur dans tout autre composants visuels où elles sont appelées. Afin que ces macros puissent être employées, **une nomenclature projet doit être bien établie** au préalable document [R4], il faudra donc s'y référer régulièrement.

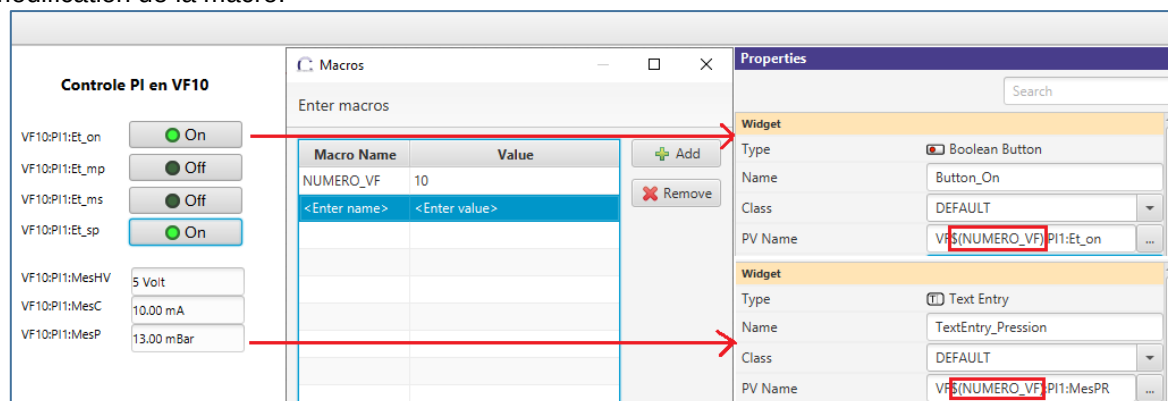
Cas concret d'utilisation d'une Macro :

Pour la définition d'une vue experte du pilotage d'une Pompe Ionique (PI) du Vide Faisceau (VF) située sur la Ligne Transport protons Titan commune. Selon la nomenclature on conviendra que toutes les variables correspondantes à cette dernière auront comme suffixe « VFXX:PI1 » où XX est à définir dans une macro selon la localisation exacte de la pompe.

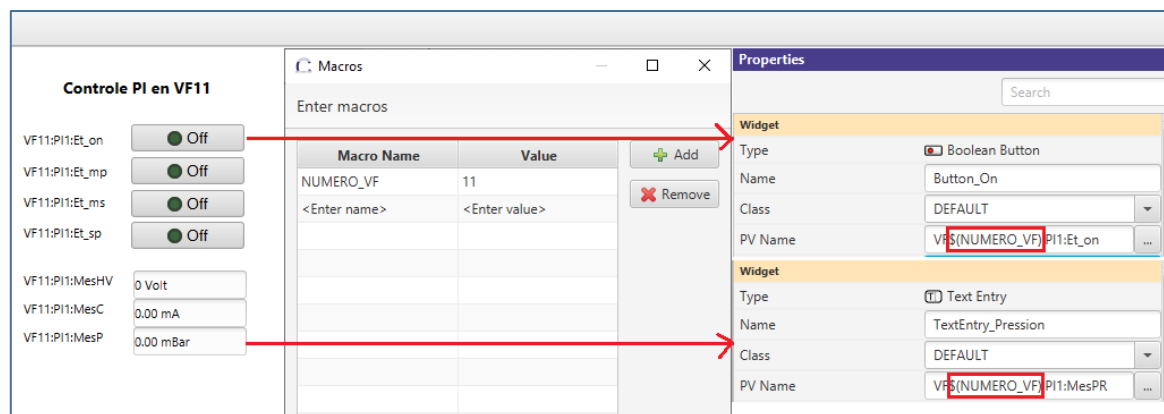
Par exemple pour une pompe en localisée en 10 :

VF10:PI1:MesHV	Mesure de la haute tension « High voltage »
VF10:PI1:MesC	Mesure du courant
VF10:PI1:MesP	Mesure de la pression
VF10:PI1:Et_on	Signal d'état en Marche
VF10:PI1:Et_mp	Signal Mode protection
VF10:PI1:Et_ms	Signal Mode pas à pas (Step)
VF10:PI1:Cd_on	Commande d'allumage
VF10:PI1:Cd_mp	Commande de mise en mode protection
VF10:PI1:Cd_ms	Commande de mise en mode pas à pas.

On peut définir une Macro « NUMERO_VF » dont la valeur sera répercutée en une seule fois sur modification de la macro.



Vue experte pour une Pompe Ionique Macro NUMERO_VF=10

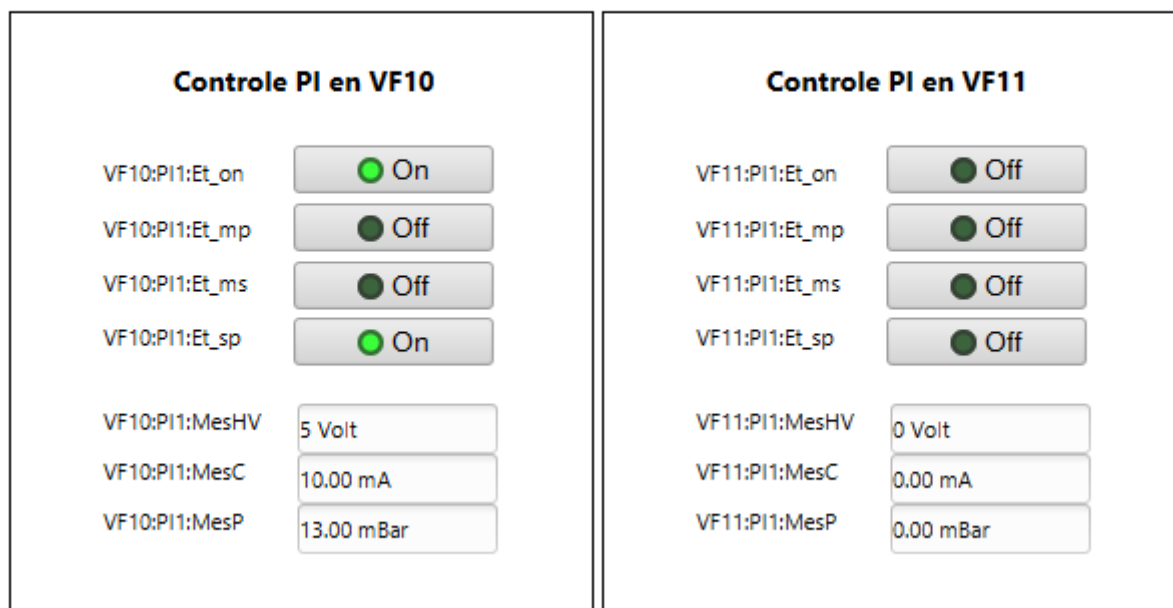


Vue experte pour une Pompe Ionique Macro NUMERO_VF=11

4.1.4 Utilisation des vues embarquées pour la modularité

Afin de rendre l'application modulaire et réutilisable, il sera recommandé d'identifier les panneaux concernant un même type d'équipement, afin de pouvoir les réutiliser dans des vues globales. Par exemple si on souhaite faire une vue qui contrôle les 2 pompes ioniques précédemment définies. On peut embarquer le panneau expert décrit en amont et modifier la valeur de la Macro « NUMERO_VF »

Contrôle des Pompes Ioniques



Utilisation d'une vue experte embarquée sur une Pompe Ionique sous Phoebus

Cette utilisation combinée des Macros et des vues embarquées, rendra la maintenance et la mise à jour efficace à long terme. Par exemple si une nouvelle variable est à contrôler pour une pompe ionique, seule la vue experte définie une seule fois est à modifier et la répercussion est automatique dans les vues qui embarquent le panneau modifié.

4.1.5 Bandeau commun et librairie commune

Avec le principe des vues embarquées, toutes les vues principales (points d'entrée des différentes applications) devront contenir un bandeau commun qui sera à disposition dans un répertoire du logiciel Phoebus. Ce bandeau commun est paramétrable par le biais de Macros. Ce dernier est composé :

- ✓ Du nom de la vue ou de l'application
- ✓ Du nom du domaine d'application par exemple VIDE ou ALIMENTATION
- ✓ Du logo de l'IRFU et de Titan
- ✓ D'un bouton Home qui ramène à la vue principale

Ce bandeau commun, sera maintenu et mis à jour depuis Phoebus et il est disponible dans différents formats. Au grand format 2550 pixels de largeur, au format moyen 1500 pixels de largeur et enfin en petit format 800 pixels de largeur. Son utilisation au travers des vues embarquées permettra aux opérateurs et utilisateurs finaux, une visualisation homogène de toutes les vues ainsi qu'une familiarisation avec ces dernières plus rapide.

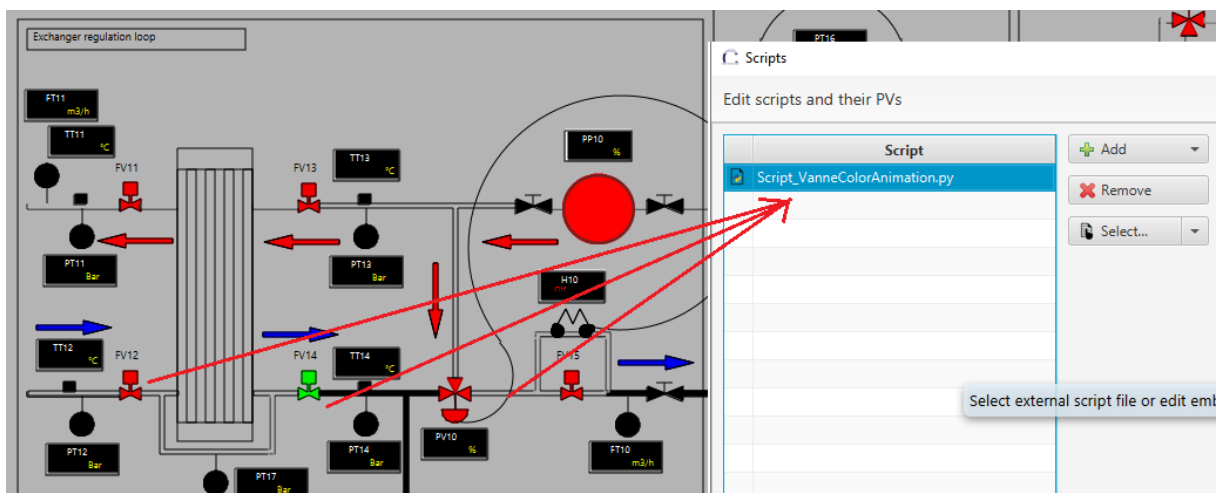


Bandeau commun paramétré pour le Vide sous Phoebus

4.1.6 Utilisation des scripts python en tant que fichier et limitation des scripts embarqués

Dans certains cas, lorsque la logique n'est pas programmable du côté d'EPICS, comme pour l'animation d'un widget avec une couleur par exemple ou bien une séquence plus complexe qu'un simple calcul, les développeurs seront contraints de développer des scripts. Phoebus offre la possibilité de faire des scripts en javascript ou en Python, aussi nous imposerons **python** qui est un langage de scripting plus répandu que javascript.

Le code de ces scripts peut être défini soit dans un composant en tant que scripts embarqués, soit dans un fichier séparé (python). Lorsque la logique programmée est réutilisable pour plusieurs composants, **on privilégiera l'utilisation de fichiers de script séparés (pas dans les composants) qu'on paramètrera avec des Macros**. Cela rendra la maintenance et la mise à jour de l'application plus efficaces. Car une modification dans le fichier aura des répercussions sur tous les composants faisant appel à ce script.



Utilisation de fichier script python sous Phoebus

4.2 BONNES PRATIQUES

4.2.1 Règles de nommage

Sous Phoebus nommer les composants selon la nomenclature définie dans le tableau ci-dessous. Cette nomenclature permet d'identifier rapidement un composant dans une vue et d'en connaître sa fonctionnalité. Cela facilite ainsi la recherche et la navigation dans l'arborescence des composants sous Phoebus. Le nom **ne doit pas contenir d'accent, ni d'espace ni de caractères spéciaux**. Il peut être défini en français ou en anglais.

Le document est maintenu, mise à jour régulièrement et archivé sur GitLab. **Ces règles de nommage ne sont pas corrélées avec la nomenclature du projet** (nom des process variables) définie en amont du projet dans un document séparé voir référence [R4]. Ces dernières n'ont aucune impacte au niveau utilisateur, elles permettent aux développeur Phoebus, un repérage rapide dans les vues, si une demande de mise à jour ou de correction est faite.

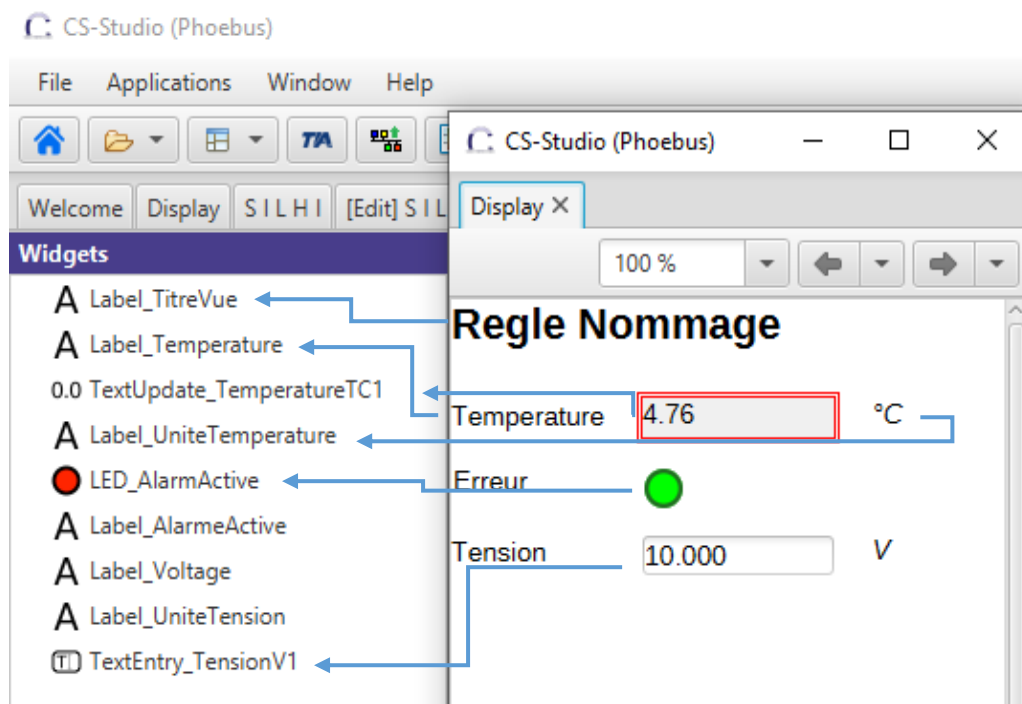
Par exemple :

Un champ d'affichage de la température connectée à une variable TC1 pourra être nommé :
[TextUpdate_TemperatureTC1](#) ou [ChampTexte_TemperatureTC1](#)

Un champ de saisi pour envoyer une tension sur une variable V1 pourra être nommé :
[TextEntry_VoltageV1](#) ou [ChampEntree_VoltageV1](#)

Type de composants	Nom du composant	Exemple
Display	\$(Metier)_vue_generale	VIDE_vue_generale
Label	Label_xxx	Label_TitreVue
Bouton	Button_xxx	Button_MarcheArret
Text Entry	Text_Entry_xxx	TextEntry_Voltage
Text Update	Text_Update_xxx	TextUpdate_Temperature
Group	Groupe_xxx	Group_SurveillanceAimant
Graph	Graph_xxx	Graph_Temperature
Combo Box	Combo_Box	ComboBox_Mode
Led	Led_xxx	Led_AlarmeActive

Règles de nommage des composants



Visualisation des composants sous Phoebus dans l'arbre des Widgets

4.2.2 Arborescence des fichiers

Afin que les vues puissent être maintenues et reprises rapidement par n'importe quelle personne familiarisée à Phoebus, on conviendra d'une arborescence de fichier commune.

Tous les fichiers projets devront être rangés dans un répertoire portant le nom de l'application. Par exemple pour le contrôle du vide faisceau de Titan « VideApplication »

Ce dernier contiendra les sous-répertoires suivants :

- **Images** : Dans lequel seront rangés les logos et images nécessaires à l'application.
- **Scripts** : Dans lequel seront rangés les scripts pythons
- **Views** : Dans lequel seront rangés les fichiers bob correspondants aux vues Phoebus.
- **EmbeddedViews** : Dans lequel seront rangés les fichiers bob des vues embarquées Phoebus

Selon le nombre de vues développées, le développeur pourra dans les répertoires Views et EmbeddedViews créer sa propre arborescence. Pour l'application du système de control principal qui doit recouvrir toutes les vues développées par les experts d'un système, les vues seront rangées dans des dossiers portant le nom de domaine concerné.

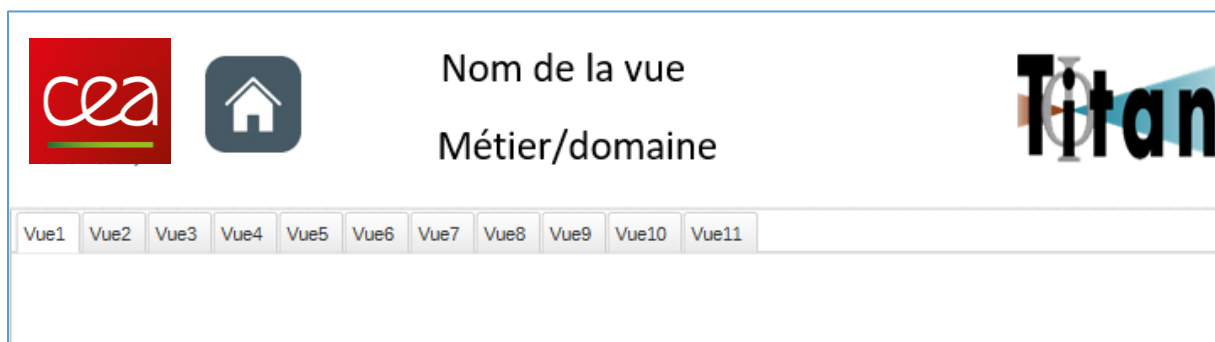
4.2.3 Navigation entre les vues

La navigation entre les vues principales d'une application se fera selon différentes manières en fonction du nombre de vues :

- Au-dessous de 10 vues, privilégier une navigation par le biais d'un menu de navigation à gauche de l'application (Comme sur ISEULT)
- Au-delà de 10 vues, privilégier une navigation par le biais d'onglet
- L'ouverture d'une petite vue experte sur un sous-élément du système, sera faite par l'ouverture d'une fenêtre de dialogue (Pop-up).



Navigation via un menu en dessous de 10 vues

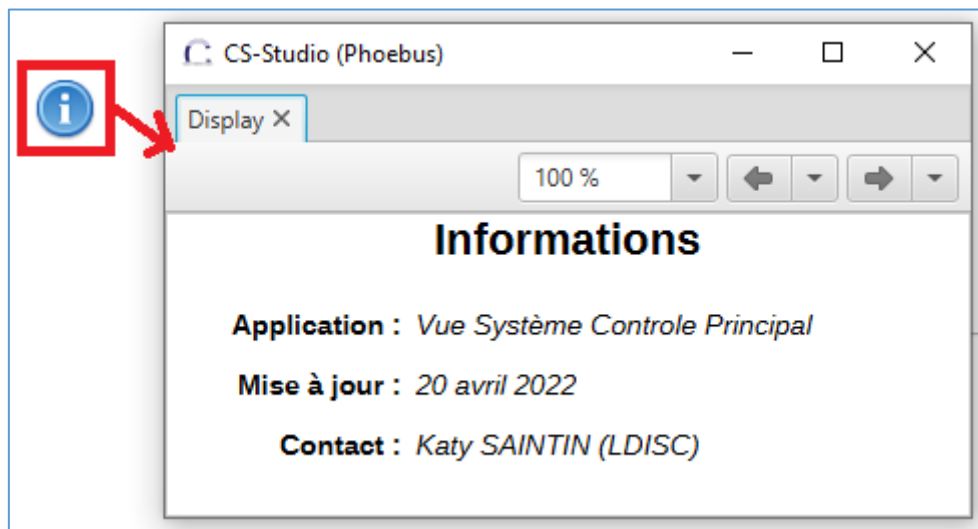


Navigation via un menu au-delà de 10 vues

4.2.4 Informations développeur et application

Il est recommandé de faire apparaître derrière un bouton informations application, les informations suivantes :

- Nom de l'application,
- Date de dernière mise à jour.
- Contact en charge de l'application.



Panneau informations

4.3 TYPOGRAPHIE, TAILLE DE POLICE ET RESOLUTION

4.3.1 Police par défaut

La police par défaut est définie dans le fichier « font.def », ce dernier est fourni dans le répertoire d'installation Phoebus/TitanCommun. Afin de le positionner comme celui par défaut, il est à ranger à la racine de l'installation Phoebus.

La police par défaut est : « Liberation Sans » en taille 14

```
// Predefined fonts that this file could re-define
Default    = Liberation Sans - regular - 14
Default Bold = Liberation Sans - bold   - 14
Header 1   = Liberation Sans - bold   - 22
Header 2   = Liberation Sans - bold   - 18
Header 3   = Liberation Sans - bold   - 16
Comment    = Liberation Sans - italic - 14
Fine Print = Liberation Sans - regular - 12
```

Fichier font.def

Police par défaut : Liberation Sans - Taille 14

Police par défaut sous Phoebus

4.3.2 Résolution d'écran

Les écrans disposés en salle de contrôle seront des écrans 16/9^{ème}; en 27pouces. Par conséquence voici les différentes tailles de vues possibles :

- Grandes : 2550x1250
- Moyenne : 1500x1250
- Petite : 800x800

Un calque commun est proposé dans les 3 tailles, afin de simplifier le développement.

4.4 CODE COULEURS

Les couleurs de fond et de textes par défaut sont définies dans le fichier « color.def », ce dernier est fourni dans le répertoire d'installation Phoebus/TitanCommun. Afin de le positionner comme celui par défaut, il est à ranger à la racine de l'installation Phoebus.

```
# Default color for text
Text=0,0,0

# Default color for 'active' text that's being edited
ActiveText=255, 255, 0

# Display background
Background = 240, 240, 240

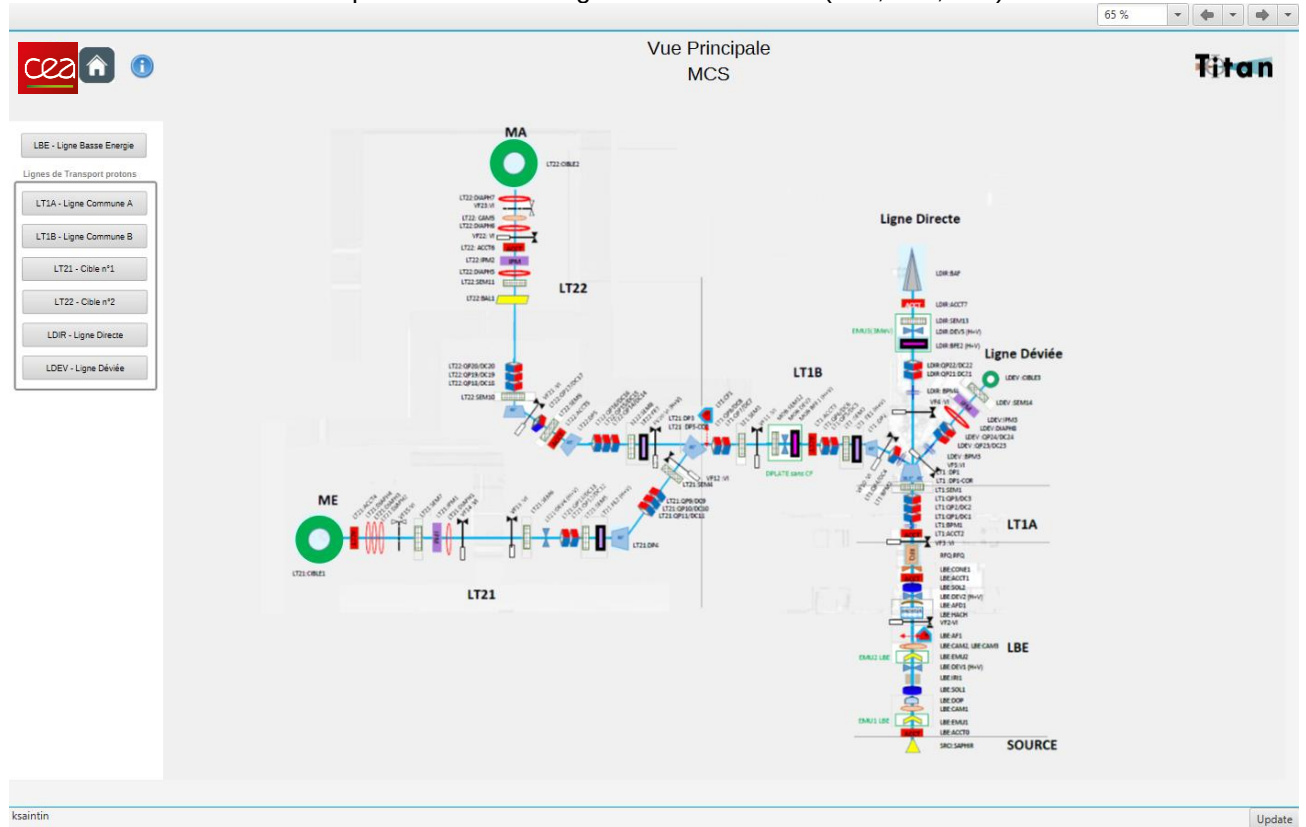
# .. for widgets that read/write a value
Read_Background = 240, 240, 240
Write_Background = 255, 255, 255

# .. for buttons
Button_Background = 210, 210, 210
```

Fichier color.def

4.4.1 Couleur de fond des vues

La couleur de fond des vues par défaut sera en gris clair. Code RVB (240, 240, 240)



Couleur de fond gris clair par défaut

4.4.2 Couleur de fond des champs de saisie des libellés et du texte.

La couleur de fond des champs texte en lecture seule est en gris clair (240,240,240)

La couleur de fond des champs texte en écriture est en blanc (255,255,255)

Les libellés textuels inactif sont transparents.

La couleur des textes est noire.

Couleur de fond PV Lecture seule : true

Couleur de fond PV écriture : 10.000

Couleur de fond des champs et des libellés

4.4.3 Code couleur des états

Après plusieurs échanges sur le sujet, nous avons arrêté le choix du code couleur sur une proposition de Robin FERDINAND. Ces 12 états sont définis dans le fichier « color.def ».

```
#Couleur Titan
MARCHE/OUVERT/HORS = 0, 255, 0, 255
ARRÊT/FERME/EN = 255,0,0,255
DEFAULT = 255,0,0,255
DECONNECTE = 255,0,255,255
EN_MOUVEMENT/ EN_COURS_DE_CHANGEMENT/INITIALISATION = 142,170,219,255
VEILLE = 255,255,0,255

#Couleur signal TOR Titan
ACTIF = 255,0,255,255
INACTIF= 127,127,127,255
```

Définition des couleurs dans le fichier « color.def »

Etat	Code couleur (R,V,B)			Commentaire
MARCHE	Vert (0, 255, 0)			
OUVERT				
HORS				
ARRÊT	Rouge (255, 0, 0)			Identique à défaut Dépend de l'application ? Proposer du rouge bordeaux
FERMÉ				
EN				
DÉFAUT	Rouge (255, 0, 0)			Clignotant / Gris foncé (127,127,127)
ALARME	Rouge	Orange	Jaune	Pas modifiable
DECONNECTÉ	Magenta (255, 0, 255)			Pas modifiable
EN MOUVEMENT	Bleu (142, 170, 219)			Peu employé
EN COURS DE CHANGEMENT				
INITIALISATION				Peu employé
VEILLE	Jaune (255, 255, 0)			Peu employé / Identique à Alarme

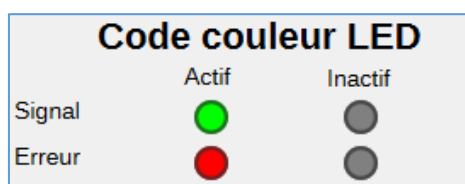
Tableau des codes couleurs et code RVB correspondant

4.4.4 Code couleur des signaux tout ou rien

On différenciera la couleur des états (animation d'un symbole sur un schéma) avec la couleur des LED correspondants à des signaux Tout ou Rien (TOR).

Type de signal => LED	Actif	Inactif
Signal	Vert	Gris foncé (127,127,127)
Erreur	Rouge	Gris foncé (127,127,127)

Tableau des codes couleurs pour les signaux TOR

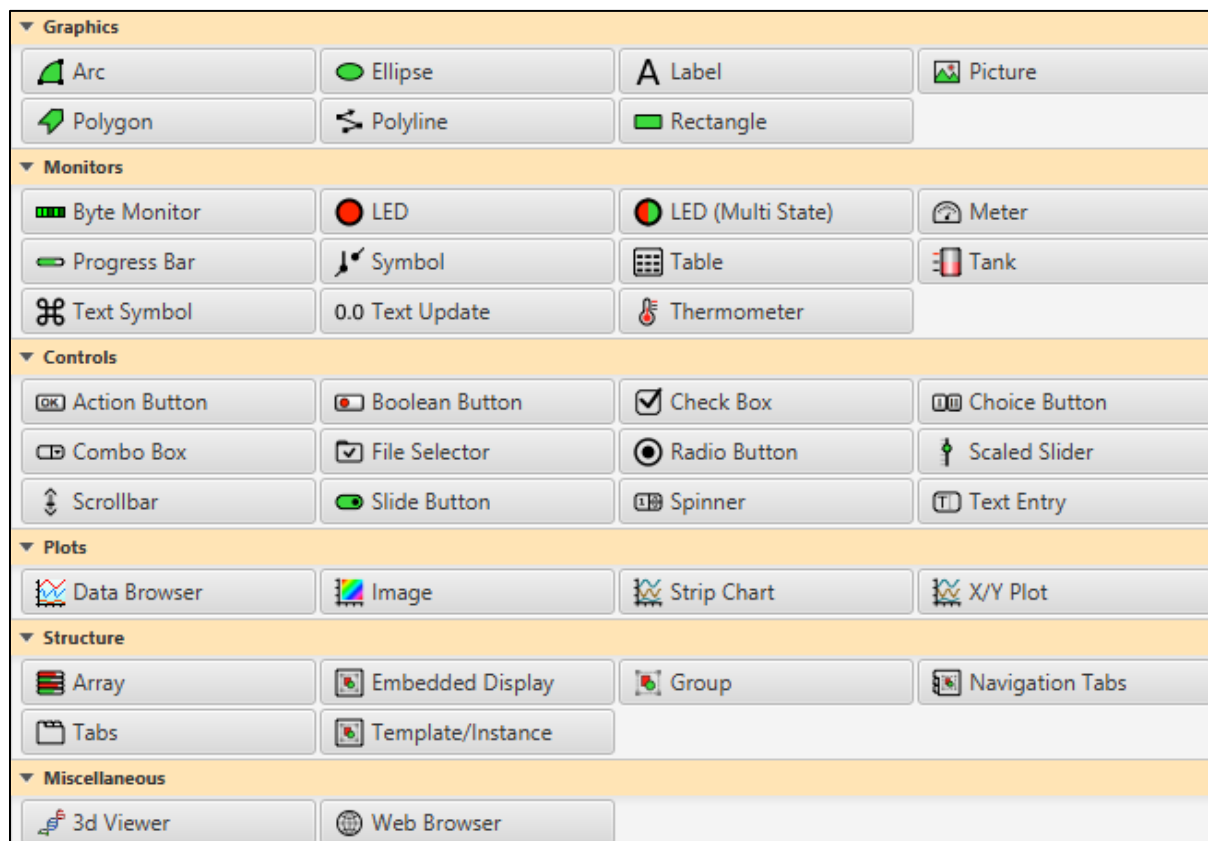


Code couleur des LED pour les signaux TOR

4.5 LIBRAIRIE DE COMPOSANTS

Phoebus fournit une liste de composants variés pour la réalisation des synoptiques de supervision. Il existe 6 types de composants :

- **Les composants graphiques** : Il s'agit de formes géométriques ou de composants statiques permettant le repérage sur un synoptique de supervision. Cependant certains composants peuvent être animé en couleur selon les états définis dans le tableau du chapitre 4.4.3 grâce à des règles d'animation.
- **Les composants de lecture (monitors)** : Il s'agit de composant permettant l'affichage de valeur d'une variable accessible en lecture seule. Ces composants ne sont généralement pas actif (Un clic sur ces derniers n'a aucun effet).
- **Les composants d'écriture (controls)** : Il s'agit de composant permettant l'envoi de consigne sur des variables accessibles en écriture. Ces composants sont actifs, la saisie numérique, le clique ou la touche « entrée » enverra la consigne au serveur.
- **Les composants d'affichage de courbes (Plots)** : Il s'agit de composant permettant l'affichage d'histogramme ou de courbes. Ces composants intègrent des fonctionnalités additionnelles pour agir sur l'affichage des courbes (modification des échelles, zoom ...)
- **Les composants de structure** : Il s'agit de composant permettant l'agencement des composants dans une vue ou bien d'affichage de tableau.
- **Les composants divers** : Il s'agit de composant d'affichage en 3D et l'intégration d'un browser web.



Librairie de composants dans Phoebus

4.6 SPECIFICATION D'UNE VUE ET PROCESSUS DE DEVELOPPEMENT

4.6.1 Spécification d'une vue

Ce document est complété par un document de spécification d'une vue « Charte graphique titan.pptx ». Ce document de spécification consiste en un fichier power point, résumant les principales recommandations de la charte graphique décrites dans ce document. Il contient également des diapositives sur lesquelles, les composants principaux de la librairie sont réutilisables. Ainsi qu'une diapositive vierge avec le calque commun de Titan paramétrable.

Ainsi il est possible de demander à un expert d'un système de réaliser une spécification de vue sans avoir à maîtriser le logiciel Phoebus.

4.6.2 Mise à jour, et gestion des versions

Afin de mettre à jour les vues embarquées (calque commun, logos ...) de manière centralisée, toutes les vues et les documents relatifs à la Charte Graphique sont versionnés sur gitlab de la DRF dans le projet : <https://drf-gitlab.cea.fr/LDISC/migration-phoebus>

GitLab intègre des fonctionnalités indispensables dans le cycle de développement logiciel pour suivre et gérer l'historique de fichiers.

Tous les documents sont stockés sur le sous projet, <https://drf-gitlab.cea.fr/LDISC/migration-phoebus/-/tree/master/TITAN>. Il contient les éléments suivants :

- Cette charte graphique ChartGraphiqueTitan.docx
- Le document de spécification Charte graphique titan.pptx
- Les ressources utiles incluant les Logos et images employés dans cette charte.
- Le fichier de définition des couleurs color.def
- Le fichier de définition de la police font.def
- Les bandeaux communs dans les 3 tailles de résolution.

4.6.3 Demande de correction et d'évolution

Pour permettre une gestion de projet efficace et anticiper le passage en mode exploitation et maintien en condition opérationnel. Le portail gitlab sert également de gestion de tickets et de demande d'évolutions.

Cela permettra une meilleure traçabilité et par conséquent un support plus efficace. Cet outil intègre un tableau de bord de suivi des tâches répondant à la méthode AGILE.

GitLab permet de définir des étiquettes correspondant à l'état d'avance d'un ticket, ainsi que la définition de jalons (milestones)

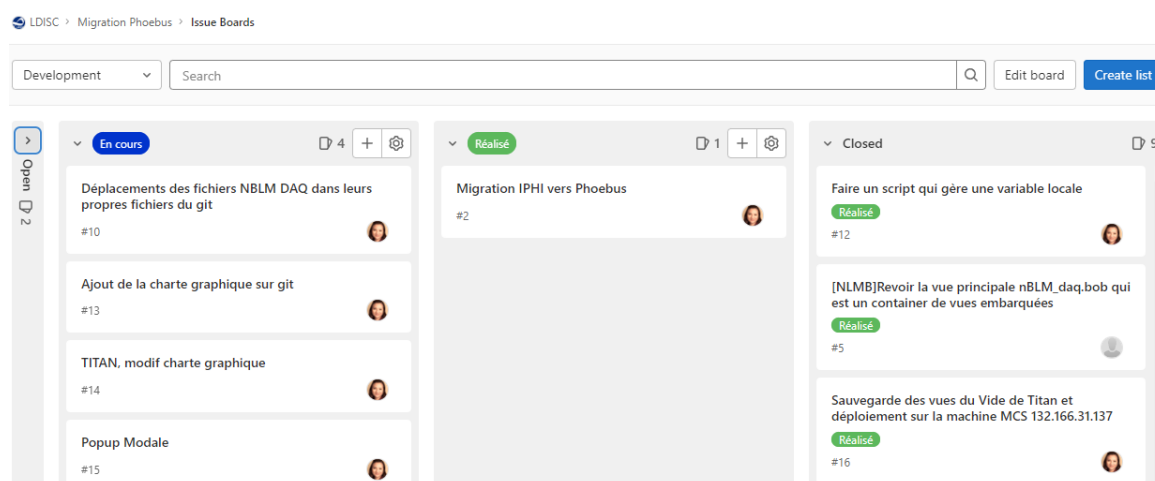


Tableau de bord de suivi des tickets GitLab (Dashboard) dans la méthodologie AGILE